README - MVP

October 19, 2026

Contents

1 Scope **2**
- 1.1 Sitemap . . . 2
- 1.2 Datamodel . . . 2
- 1.3 Curation views . . . 2
- 1.4 Data access . . . 2
- 1.5 Out-of-scope . . . 2

2 Stakeholders **3**

3 Milestones after MVP **3**
- 3.1 Stable endpoints for REST/SPARQL . . . 3
- 3.2 Minimal drupal distribution . . . 3
- 3.3 Full data input of all STI resources . . . 3
- 3.4 User-scoped curation workflow views . . . 3
- 3.5 Tested&confirmed ATIO data model in system . . . 3
- 3.6 Addtional curation workflows . . . 3
- 3.7 source record -> atio innovation derivation by AI . . . 3
- 3.8 Content display portal -> open social . . . 3
- 3.9 role mapping open social <-> portal . . . 3

4 Documentation of features **3**
- 4.1 REST API and GET access DATALAB . . . 3
  - 4.1.1 GET: Collection-scoped content . . . 4
- 4.2 Drupal: `source_record` -> `atio_innovation` derivation and AI-enrichment 4
  - 4.2.1 **TODO** Next steps ATIO . . . 4
- 4.3 Collection / group - scoped permissions handling . . . 5

5 Documentation of custom modules **5**
- 5.1 atio . . . 5
- 5.2 `import_ird_json` . . . 5
- 5.3 `sti_rdf` . . . 5
  - 5.3.1 **TODO** Next steps . . . 5
- 5.4 `sti_graph_explorer` . . . 5
- 5.5 `sti_dedupe` . . . 5
  - 5.5.1 **TODO** Next steps . . . 6
- 5.6 `sti_portal_api` . . . 6
  - 5.6.1 **TODO** Next steps . . . 6

1 Scope

The MVP shows the proof that a joinup-based STI portal will be able to achieve the project aims as defined in the CAPEX proposal and the ATIO design doc. It will deliver:

1.1 Sitemap

- Welcome: Landing page with access to ATIO and STI collections
- Collection pages
  - Faceted search
  - Customization of about, logo, banner
  - `source_record` (> 100) and `atio_innovation` (2-3)

1.2 Datamodel

- `source_record`: fully modeled in both RDF and SQL
- `atio_innovation`: fully modeled in both RDF and SQL
- `collection`: fully modeled in both RDF and SQL
- `claim`: fully modeled in both RDF and SQL

1.3 Curation views

Curators will be able to solve

- deduplication of `source_record`
- update of a semantic map using skos and projected to the triplestore
- editing of all 4 types of the current data model in dedicated views
- QA review of relevant content

1.4 Data access

- `source_record` will be readable using REST
- all data will be accessible using SPARQL
- `atio_innovation` will be writeable using REST

1.5 Out-of-scope

- ECAs / algorithms transforming `source_record` into `atio_innovation`
- Full UX/style copy of current site (e.g. ATIO landing page)
- Innovation family has to be actually figured out by ATIO
- Integration with open social will be built later
- Joinup pruning / new distribution generation will happen later

2 Stakeholders

Audience	Expectation
Valeria/Diego	1. Ontology is implemented as closely as possible, especially when it comes to atio-innova
Datalab	Fixed contract of data input format & APIs (both rest and sparql)
Henry	We can actually give everyone (collection) their own landing page, and curation space
Alessia	I can do my blueprint work (register innovations in 'Roadmap' following taxonomy define

3 Milestones after MVP

3.1 Stable endpoints for REST/SPARQL

DEADLINE *<2026-10-29 gio>*

3.2 Minimal drupal distribution

3.3 Full data input of all STI resources

- depends on distribution

3.4 User-scoped curation workflow views

DEADLINE *<2026-10-29 gio>*

3.5 Tested&confirmed ATIO data model in system

DEADLINE *<2026-11-12 gio>*

3.6 Addtional curation workflows

3.7 source record -> atio innovation derivation by AI

DEADLINE *<2026-12-31 gio>*

3.8 Content display portal -> open social

DEADLINE *<2026-11-12 gio>*

3.9 role mapping open social <-> portal

4 Documentation of features

All documentation also lives in the respective modules, a copy for context below

4.1 REST API and GET access **datalab**

Joinup uses the `rest` module (config: /admin/config/services/rest) to define REST endpoints one-by-one. I prefer this over the REST-JSON module, because that one opens REST endpoints to any node in the system. The preferred control to only open access to specific resources makes more sense from a security perspective.

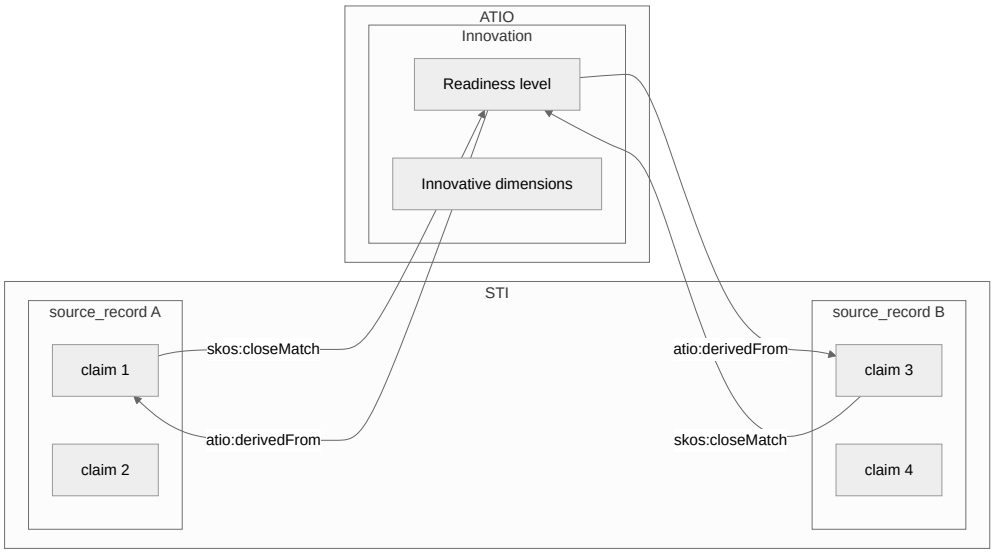
4.1.1 GET: Collection-scoped content

The relevant GET endpoint is: `/api/collection-solutions/v1/{collection}: GET`. It is defined by the `sti_portal_api` module. It retrieves all content for a given collection. Filters are:

- `?moderation=|all|published|unpublished|` : published status. Default: published
- `?type=|source_record|atio_innovation|` : which content types, comma-separated
- `?include=claims` : whether or not to include the claim information. Default: false
- `?limit=N` : set a maximum amount to content to retrieve. This copies the pagination on the collection, so a page needs to be speciefied. Default: no limit
- `?page=P` : page through results. Default: all pages

4.2 Drupal: `source_record` -> `atio_innovation` derivation and AI-enrichment

Improving the traceability, quality and modularity of the creation of the `atio_innovation` is the core reason for the creation of this MVP. While the STI namespace now handles data input of any strucuture, it results in collections that each present data in their own, unique way. The important step is standardization: The derivation of comparable innovation records requires a centralized format. This is provided by the ATIO team; the only thing changing is the input format of the data they work with to derive this standardized view.

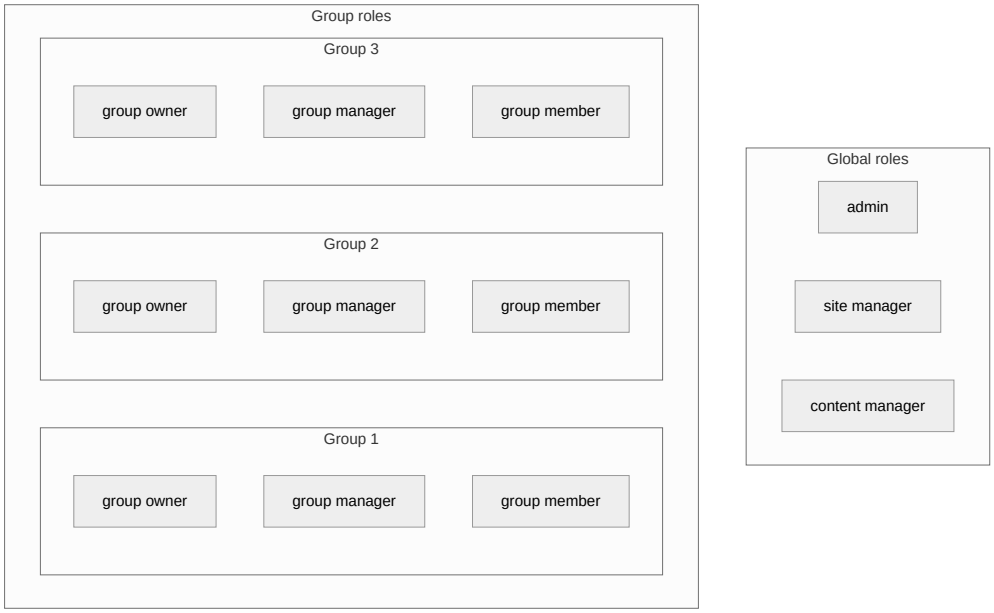


4.2.1 TODO Next steps atio

Valeria has created a rather complex claimResolution model. Since historically all `atio_innovation` generation has been done by AI and the fact that a single collection easily provides hundreds of thousands of claims, I think claim to `atio_innovation` matching will not be done manually. That means we likely will need a more pragmatic claim resolution. Simply by exclusion of `atio:derivedFrom` statements, decisions of which content is used to fill an `atio_innovation` fields are implicitly recorded. In Drupal, this could be modeled as a paragraph [`field_derievd_from`, `explanation`] of which only the

claim derivation is synced to the triplestore, while the explanation plain text is recorded Drupal-side for potential human revision.

4.3 Collection / group - scoped permissions handling



5 Documentation of custom modules

5.1 atio

This module is a port of the existing core features of the current STI portal implementation. This is mainly a list of content type entities and taxonomy vocabularies. The main difference is a clearer namespace separation to make clear which content belongs to the ATIO work, and which is simply imported to the platform ('sti').

5.2 `import_ird_json`

5.3 `sti_rdf`

5.3.1 TODO Next steps

- The README.md mentions that this module is responsible for rendering claim facets, but I thought the code in `import_ird_json` is the relevant one for this task?

5.4 `sti_graph_explorer`

5.5 `sti_dedupe`

Deduplication is an important task an registered as not fully resolved in the current ATIO design document. It could be done at the `source_record` or the `atio_innovation` level. This module solves it at the `source_record` level. The rationale is that data input from many different sources might likely contain duplicates. Registering this early will allow to map potentially differing claims from different collections about the same `source_record` possible. It also makes the derivation of an `atio_innovation` easier. Technically this is solved by another field on the claims paragraph: `duplicate_of` has claimedAt and claimedBy fields. This also solves the issue of agreement: If collection

A claims `source_record` A is a duplicate of `source_record` B from collection B this statement is recorded. Only if collection B also makes the reverse claim do we have a confirmed agreement. The module provides an easy interface for data curators to register this duplicate claim.

5.5.1 TODO Next steps

- Role-scoping: Global admin roles should be able to make any claims, collection owners and facilitators can only claim uni-directionally.
- Role-scoping: The interface should only show potential duplicates to resolve scoped by the current user's role
- There is already a whole section on claim resolution, but I haven't decided yet how to handle it. It also relies on fields that don't exist yet

5.6 `sti_portal_api`

This module provides the API endpoint to read a collection's content. It should be the main entrypoint to the system that is being built by the datalab colleagues to get content data. Ideally it is used in conjunction with sparql queries that provide semantic context to the content. For the data retrieval it is assumed that it is done per-collection and that the user only wishes to filter by publication status, content type and whether or not to add the detailed claims or to only return the content core fields.

- `?moderation=|all|published|unpublished|` : published status. Default: published
- `?type=|source_record|atio_innovation|` : which content types, comma-separated
- `?include=claims` : whether or not to include the claim information. Default: false
- `?limit=N` : set a maximum amount to content to retrieve. This copies the pagination on the collection, so a page needs to be speciefied. Default: no limit
- `?page=P` : page through results. Default: all pages

5.6.1 TODO Next steps

Access needs to be aligned with the future roles&permissions model. Ideally, a future version would allow the same group scoping that should exist for the website: Global admin roles can access & edit any content, but groups also have an owner, facilitator and member scope, where a groups owner and facilitator have read/write access to only their own group's content.